



Software Engineering

LECTURE 9: SOFTWARE TESTING

What is Software Testing?

- Testing is intended to show that a program does what it is intended to do
- Discover program defects before it is put into use.
- To achieve this, you execute the program with artificial data
- You check the results of the test run for errors, anomalies or information about program's non-functional attributes.

Why Testing and Analysis?

Software is **never correct** no matter which developing technique is used

Any software **must be verified**.

Software testing and analysis are:

- important to **control the quality** of the **product** (and of the **process**)
- very (often too) **expensive**
- difficult and stimulating



Ariane 5

Development took 10 years & \$7 billion

On June 4, 1996, the flight of the Ariane 5 launcher ended in a failure

Only about 37 seconds after initiation of the flight sequence, at an altitude of about 3,700 m, the launcher veered off its flight path, broke up and exploded

“Fixed” Ariane 5 – currently being used



Ariane 5 – Root Cause

Source: ARIANE 5 Flight 501 Failure, Report by the Inquiry Board

A program segment for converting a floating point number to a signed 16 bit integer was executed with an input data value of a 64-bit number.

The segment was written for Ariane 4 and the conversion was left unprotected, because it was thought that Ariane 4 will never attain such a value.

The subsystem containing this segment was reused for Ariane 5 as it had been successfully used over years for Ariane 4

This run time error (out of range, overflow), which arose in both the active and the backup computers at about the same time, was detected and both computers shut themselves down.

This resulted in the total loss of altitude control. This breakup was detected by an on-board monitor which ignited the explosive charges to destroy the vehicle in the air.

Ironically, this program segment was not required by Ariane 5

Ariane 5 – Lessons Learned

Adequate exception handling and redundancy strategies (real function of a backup system, degraded modes?)

Clear, complete, documented specifications (e.g., preconditions, post-conditions)

But perhaps more importantly: usage-based testing (based on operational profiles), in this case actual Ariane 5 trajectories

Note this was not a complex, computing problem, but a deficiency of the software engineering practices in place ...



F-18 crash

An F-18 crashed because of a missing exception condition:

- An if ... then ... block without the else clause that was thought could not possibly arise.

In simulation, an F-16 program bug caused the virtual plane to flip over whenever it crossed the equator, as a result of a missing minus sign to indicate south latitude.



Fatal Therac-25 Radiation

In 1986, a man in Texas received between 16,500-25,000 radiations in less than 10 sec, over an area of about 1 cm.

He lost his left arm, and died of complications 5 months later.

Therac 25

Incident: A patient passed away

Failure: The device applied higher frequency of radiations than what was safe. Safety range: [1...10,000 Hz]

Fault: The software controller of the device did not have a conditional block (if else statements) to perform range checking on the frequency of the radiation to be applied.

Errors:

- 1. The SW developer of the device controller system had forgotten to include a range checking conditional block on the frequency of the radiation to be applied.
- 2. The device operator was NOT supposed to enter anything outside [1...10,000 Hz] range.

Errors

Errors are part of our daily life

Humans make errors in their thoughts, actions, and in the products that might result from their actions

Errors occur wherever humans are involved in taking actions and making decisions

These fundamental facts of human existence make testing an essential activity

Faults and Failures

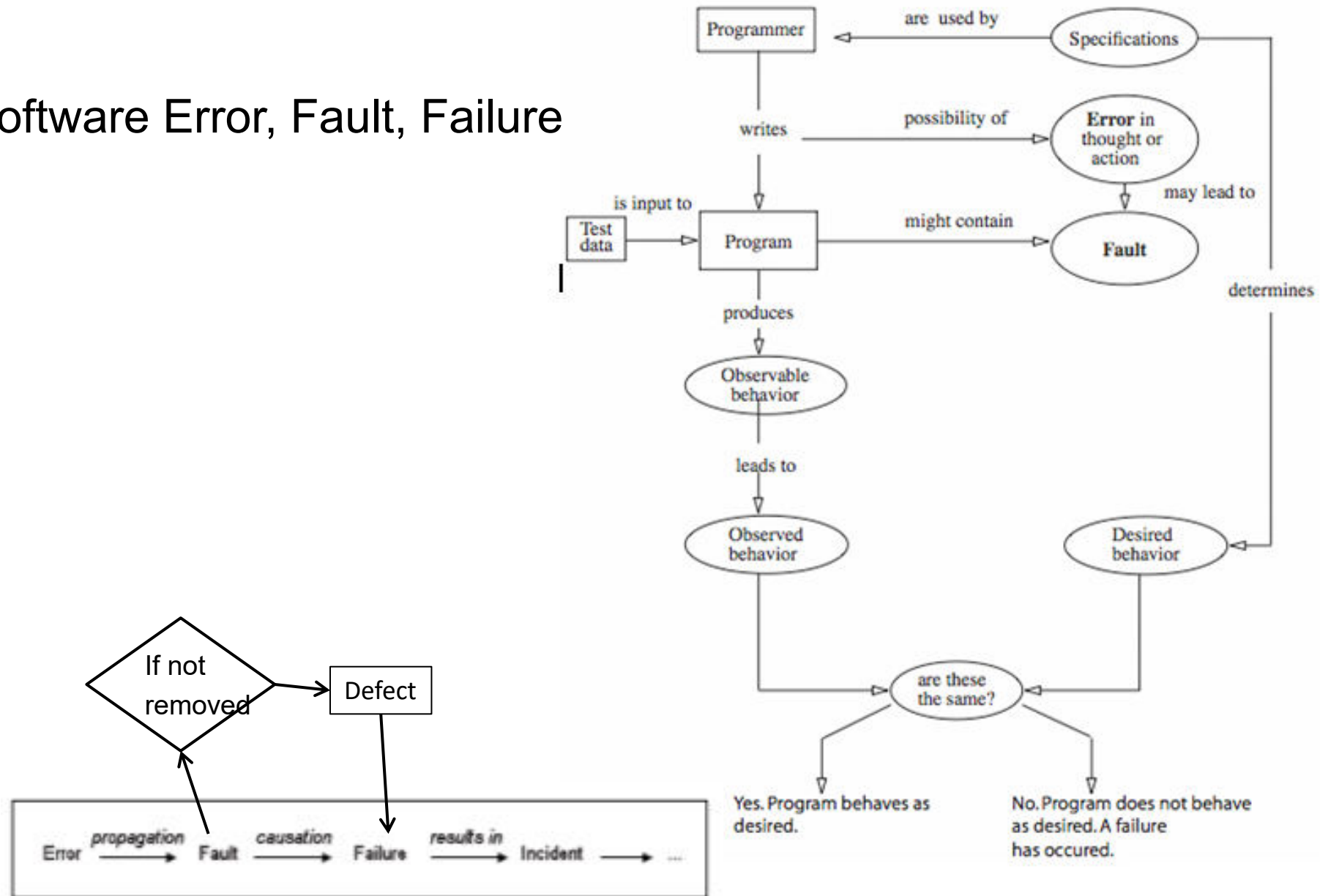
Fault (bug):

Programming or design error whereby the delivered system does not conform to specification (e.g., coding error, protocol error)

Failure:

Software does not deliver the service expected by the user (e.g., mistake in requirements, confusing user interface)

Software Error, Fault, Failure



Terminology

Fault avoidance

Build systems with the objective of creating fault--free (bug--free) software.

Fault detection (testing and verification)

Detect faults (bugs) before the system is put into operation or when discovered after release.

Fault tolerance

Build systems that continue to operate when problems (bugs, overloads, bad data, etc.) occur.

Test Cases

A test case, in software engineering, is a set of conditions or variables under which a tester will determine whether an application, software system or one of its features is working as it was originally established for it to do.

***“A set of test inputs, execution conditions, and expected results developed for a particular objective, such as to exercise a particular program path or to verify compliance with a specific requirement.”
(IEEE)***

Developing and writing Test Cases is one of the major and most important activity in the entire testing cycle of an application

How to write good test cases

When you begin writing test cases, there are a few steps that you need to follow to ensure that you are writing good test cases

Identify the purpose of testing

- You need to understand the requirements to be tested. Writing test cases for a module requires that you understand the features/user requirements for that module

Define how to perform the testing

- This involves developing typical scenarios of use

Identify any non-functional requirements

- Understand other aspects of the software related to non-functional requirements

Structuring you Test Cases

1. Test Case ID (This is the unique number which helps in identifying a specific test case)
2. Test Case Description
3. Module to be tested (Usually we provide Requirement ID to maintain traceability between test case and requirements)
4. Test Data (We provide variable and values based on need of the test case)
5. Test Steps (Steps to be executed)
6. Expected results (How application should behave after performing stated test steps)
7. Actual results (Actual output tester will get after preforming steps)
8. A result (Pass or fail after comparing expected and actual results)
9. Comments (We can provide screen shot or any other relevant information to help developer debug the code)

R-001:

The users should be able to add two numbers and view their result on the display.

Use Case: <u>UC01</u>	Add Two Numbers
Actors:	User
Purpose:	Add two numbers and view their result
Overview:	The user inputs two numbers and (then adds them and) checks the result, displayed on the screen.
Type:	Primary, Real
Cross References:	R-001

Typical Course of Events

Actor Action	System Response
1. The actor opens the calculator. The keypad and display screen appears.	
2. The actor input the first number by clicking on the keypad or using keyboard.	3. The digit is displayed on the screen.
4. The actor clicks or presses the “+” key.	
5.The actor then adds the second number as (2).	6. The pressed digit is displayed on the screen.
7. The actor clicks the “=” key.	8. The sum of the two digits is displayed on the screen.

USE CASE FOR ADDING NUMBERS

Test Case ID: *T-101* Test Item: *Add Numbers*
Wrote By: (*tester name*) *Junaid* Documented Date: *26th April 2005*
Test Type: *Manual* Test Suite#: *NA*
Product Name: *Windows Calculator* Release and Version No.: *V 1.0*
Test case description:
 Add any two Numbers
Operation procedure:
 Open Calculator
 Press "1"
 Press "+"
 Press "2"
 Press "="
Pre-conditions: Post-conditions:
Calculator Opened *Result Displayed*
Inputs data and/or events: Expected output data and/or events:
1 + 2 = *3*
Required test scripts (*for auto*): *NA*
Cross References: (*Requirements or Use Cases*) *R-001, UC-01*

Test CASE FOR ADDING NUMBERS

Testing Strategies

SW testing is one element of a broader topic referred to as V&V (many SQA activities).

- ❑ **Verification** (correctly implement specific function)
 - Defect **detection** and **correction**
 - **Comparison** between **implementation** and the corresponding **specification**
 - Are we building the **product right**?
- ❑ **Validation** (SW is traceable to customer requirements)
 - Defect **prevention**
 - Provision of sound basis for **specific design decisions**
 - Are we building the **right product**?

Example: **Checking the printing of receipts** is **verification** and **correct printing (info)** is **validation**.

Types/Strategies of testing

Code Inspections

Software (Module) Testing

- Unit Testing
- Functional Testing

Integration Testing

- Compliance
- Interoperability Testing

System Testing

- Recovery
- Security
- Environment

Acceptance Testing and Release Testing

Regression Testing

Stress, Security, performance, Load Testing etc

Software Testing

- ◆ A better definition: "the process of running software with the intent of finding an error".
- ◆ The way most testing works is to input a set of values, then compare the expected result with the out comes/computed ones.
- ◆ Depending on the type of data input, we can identify three levels of program correctness possible, probable, and absolute correctness.

Three Levels of Correctness

Possible correctness

- ◆ Obtaining **correct output** for some **arbitrary input** (single set).
- ◆ If the outcome is **wrong**, the program **cannot possibly be correct**.
- ◆ For example a multiply program: for input 2 and 3, if result is 6; is **possibly correct**.

Three Levels of Correctness

Probable correctness

- ◆ Obtaining correct output for a number of carefully selected inputs.
- ◆ If all potential problematic areas are checked in this way, the program is probably correct.
- ◆ Try several values, including the obvious “problem” values such as zero, largest negative, 1, -1 and so on.

Three Levels of Correctness

Absolute correctness

- ◆ can be demonstrated **only by a test** that involves **every possible combination of inputs**.
- ◆ Requires **huge amount of time**; it is therefore **not practical** {However for some programs we can prove correctness mathematically}.
- ◆ Imagine the same above test for a 32 bit machine (how many combinations are possible? (Hundreds of Millions)).

A Small Testing Example

You are to develop a small program that reads three integer values from an input dialog. The three values represent the lengths of the sides of a triangle.

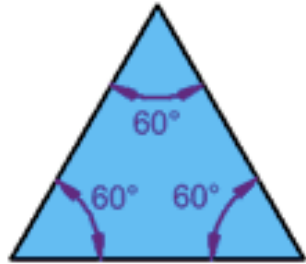
The program displays a message that states whether the **triangle is scalene, isosceles, or equilateral**.

Write a set of test cases—specific sets of data—to properly test the program

- The set of test data must be handled by the program correctly to be considered a successful program.

Remember that

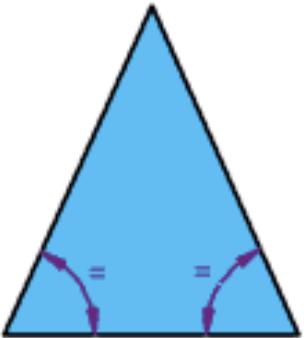
- a **scalene** triangle is one where **no two sides are equal**,
- whereas an **isosceles** triangle has **two equal sides**, and
- an **equilateral** triangle has **three sides of equal length**.



Equilateral Triangle

Three equal sides

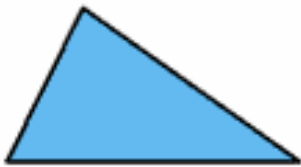
Three equal angles, always 60°



Isosceles Triangle

Two equal sides

Two equal angles



Scalene Triangle

No equal sides

No equal angles

Definitions

```
int main()
{
    float a,b,c;
    cin>>a >> b >>c;
    if(a<(b+c) && b < (a+c) && c< (a+b))
    {
        cout<<"\nIt is a Triangle.";
        if (a==b && b==c)
            cout<<"\nIt is a Equilateral Triangle.";
        if(a==b || a==c || b==c)
            cout<<"\nIt is a Isosceles Triangle.";
        else
            cout<<"\nIt is a Scalene Triangle.";
    }
    else
        cout<<"This Triangle is not possible.";

    return 0;
}
```

WHAT TEST CASES DO YOU THINK SHOULD BE THERE?

For the input values to represent a triangle, they must be integers greater than 0 where the sum of any two is greater than the third.

Questions to Answer ??

1. Do you have a test case that represents a valid scalene triangle
2. Do you have a test case that represents a valid equilateral triangle?
3. Do you have a test case that represents a valid isosceles triangle?
4. Do you have at least three test cases that represent valid isosceles triangles such that you have tried all three permutations of two equal sides (such as, 3, 3, 4; 3, 4, 3; and 4, 3, 3)?
5. Do you have a test case in which one side has a zero value?
6. Do you have a test case in which one side has a negative value?
7. Do you have a test case with three integers greater than zero such that the sum of two of the numbers is equal to the third? (That is, if the program said that 1, 2, 3 represents a scalene triangle, it would contain a bug.)

Questions to Answer ?? (2)

- 8 Do you have at least three test cases in category 7 such that you have tried all three permutations where the length of one side is equal to the sum of the lengths of the other two sides (for example, 1, 2, 3; 1, 3, 2; and 3, 1, 2)?
- 9 Do you have a test case with three integers greater than zero such that the sum of two of the numbers is less than the third (such as 1, 2, 4 or 12,15,30)?
- 10 Do you have at least three test cases in category 9 such that you have tried all three permutations (for example, 1, 2, 4; 1, 4, 2; and 4, 1, 2)?
- 11 Do you have a test case in which all sides are zero (0, 0, 0)?
- 12 Do you have at least one test case specifying noninteger values (such as 2.5, 3.5, 5.5)?
- 13 Do you have at least one test case specifying the wrong number of values (two rather than three integers, for example)?
- 14 For each test case did you specify the expected output from the program in addition to the input values?

Testing the triangle program – Some thoughts!

A very simple program

To do exhaustive testing

- Test with all possible values of a, b, & c (the three sides)
 - And their combinations
 - 32-bit integer, 2^{32} possible values for one integer
 - Assuming only integers from 1 to 10, there are 10^{12} possible values for a triangle.
 - Testing 1000 cases per second, you would need 317 years!
- Test with all the invalid inputs
 - e.g., All strings ;)

Testing the triangle program – Some thoughts!

If testing a trivial program is so complex, what about testing large programs of thousands of lines of code, e.g., air traffic control software

Solution?

Testing the triangle program – Some thoughts!

If testing a trivial program is so complex, what about testing large programs of thousands of lines of code, e.g., air traffic control software

Solution?

- Use of sophisticated testing techniques
 - Select the test cases that have most chances of triggering a failure
 - e.g, boundary values

Test case design

OBJECTIVE ---- to uncover errors

CRITERIA ----- In a complete manner

CONSTRAINT ----- with a minimum of effort and time

- ◆ A **successful test** -- -- that uncovers an as-yet undiscovered error.
- ◆ **Minimum number of required tests** with 100% functional coverage and 0% redundancy.
- ◆ Rich variety of **test case design methods**
 - Cause-effect graphing, Equivalence Class Partitioning, Boundary Analysis, and vendor specific: client/server, OO test case design.

Possible approaches:

- (1) Knowing the **specified functions**, tests can be conducted to demonstrate that each **function is fully operational (Black Box)**.
- (2) Knowing the **internal workings** of a product, tests can be conducted to ensure that “all gears mesh“ (**White Box**).

**"Bugs lurk in corners
and congregate at
boundaries ..."**

Boris Beizer

